LAPORAN PRAKTIKUM
Struktur Data

MODUL 6
C++



Disusun oleh:

Rafli Firmansyah
109082500095
S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
TAHUN 2026

Guided

1. Guided 1 : dll.h

Source Code

```
#ifndef DLL_H
#define DLL_H

#include <iostream>
#include <string>
using namespace std;

struct Kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};
struct ElmList {
    Kendaraan info;
    ElmList* next;
    ElmList* prev;
};

// STRUKTUR LIST
struct List {
    ElmList* first;
    ElmList* last;
};

// FUNGSI DASAR
void createList(List &L);
ElmList* alokasi(Kendaraan x);
void dealokasi(ElmList* &P);
void printInfo(List L);
ElmList* findElm(List L, string nopol);

// FUNGSI INSERT
void insertFirst(List &L, ElmList* P);
void insertLast(List &L, ElmList* P);
void insertAfter(List &L, ElmList* Prec, ElmList* P);
void insertBefore(List &L, ElmList* Succ, ElmList* P);

// FUNGSI DELETE
void deleteFirst(List &L, ElmList* &P);
void deleteLast(List &L, ElmList* &P);
void deleteAfter(List &L, ElmList* Prec, ElmList* &P);
void deleteBefore(List &L, ElmList* Succ, ElmList* &P);

#endif
```

2. Guided 2 : dll.cpp

Source Code

```
#include "dll.h"

// MEMBUAT LIST KOSONG
void createList(List &L) {
    L.first = nullptr;
    L.last = nullptr;
}

// MEMBUAT NODE BARU
ElmList* alokasi(Kendaraan x) {
    ElmList* P = new ElmList;

    P->info = x;
    P->next = nullptr;
    P->prev = nullptr;

    return P;
}

// MEMBEBASKAN MEMORI NODE
void dealokasi(ElmList* &P) {
    if (P != nullptr) {
        delete P;
        P = nullptr;
    }
}

// MENAMPILKAN SEMUA DATA KENDARAAN
void printInfo(List L) {
    ElmList* P = L.first;

    if (P == nullptr) {
        cout << "List kosong." << endl;
        return;
    }

    while (P != nullptr) {
        cout << "Nopol    : " << P->info.nopol << endl;
        cout << "Warna    : " << P->info.warna << endl;
        cout << "Tahun    : " << P->info.thnBuat << endl;
        cout << "-------------------" << endl;

        P = P->next;
    }
}

// MENCARI NODE BERDASARKAN NOMOR POLISI
ElmList* findElm(List L, string nopol) {
    ElmList* P = L.first;
```

```
    while (P != nullptr) {
        if (P->info.nopol == nopol) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

// INSERT FIRST: MENAMBAHKAN NODE DI AWAL
void insertFirst(List &L, ElmList* P) {
    if (P == nullptr) {
        return;
    }

    P->prev = nullptr;
    P->next = L.first;

    if (L.first != nullptr) {
        L.first->prev = P;
    } else {
        L.last = P;
    }

    L.first = P;
}

// INSERT LAST: MENAMBAHKAN NODE DI AKHIR
void insertLast(List &L, ElmList* P) {
    if (P == nullptr) {
        return;
    }

    P->next = nullptr;
    P->prev = L.last;

    if (L.last != nullptr) {
        L.last->next = P;
    } else {
        L.first = P;
    }

    L.last = P;
}

// INSERT AFTER: MENAMBAHKAN NODE SETELAH NODE ACUAN
void insertAfter(List &L, ElmList* Prec, ElmList* P) {
    if (Prec == nullptr || P == nullptr) {
```

```
        return;
    }

    if (Prec == L.last) {
        insertLast(L, P);
        return;
    }

    P->next = Prec->next;
    P->prev = Prec;

    Prec->next->prev = P;
    Prec->next = P;
}

// INSERT BEFORE: MENAMBAHKAN NODE SEBELUM NODE ACUAN
void insertBefore(List &L, ElmList* Succ, ElmList* P) {
    if (Succ == nullptr || P == nullptr) {
        return;
    }

    if (Succ == L.first) {
        insertFirst(L, P);
        return;
    }

    P->next = Succ;
    P->prev = Succ->prev;

    Succ->prev->next = P;
    Succ->prev = P;
}

// DELETE FIRST: MENGELUARKAN NODE PERTAMA
void deleteFirst(List &L, ElmList* &P) {
    if (L.first == nullptr) {
        P = nullptr;
        return;
    }

    P = L.first;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.first = P->next;
        L.first->prev = nullptr;
    }

    P->next = nullptr;
```

```
    P->prev = nullptr;
}

// DELETE LAST: MENGELUARKAN NODE TERAKHIR
void deleteLast(List &L, ElmList* &P) {
    if (L.last == nullptr) {
        P = nullptr;
        return;
    }

    P = L.last;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.last = P->prev;
        L.last->next = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE AFTER: MENGELUARKAN NODE SETELAH NODE ACUAN
void deleteAfter(List &L, ElmList* Prec, ElmList* &P) {
    if (Prec == nullptr || Prec->next == nullptr) {
        P = nullptr;
        return;
    }

    P = Prec->next;

    if (P == L.last) {
        deleteLast(L, P);
        return;
    }

    Prec->next = P->next;
    P->next->prev = Prec;

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE BEFORE: MENGELUARKAN NODE SEBELUM NODE ACUAN
void deleteBefore(List &L, ElmList* Succ, ElmList* &P) {
    if (Succ == nullptr || Succ->prev == nullptr) {
        P = nullptr;
        return;
    }
```

```
    P = Succ->prev;

    if (P == L.first) {
        deleteFirst(L, P);
        return;
    }

    Succ->prev = P->prev;
    P->prev->next = Succ;

    P->next = nullptr;
    P->prev = nullptr;
}
```

3. Guided 3 : main.cpp

Source Code

```
#include "dll.h"

int main() {
    List L;
    createList(L);

    Kendaraan K1 = {"D001", "Merah", 2022};
    Kendaraan K2 = {"D002", "Hitam", 2023};
    Kendaraan K3 = {"D003", "Biru", 2024};
    Kendaraan K4 = {"D004", "Abu-abu", 2025};
    Kendaraan K0 = {"D000", "Putih", 2021};

    // INSERT LAST: MEMASUKKAN D001 DI AKHIR
    ElmList* P = alokasi(K1);
    insertLast(L, P);

    // INSERT LAST: MEMASUKKAN D002 DI AKHIR
    P = alokasi(K2);
    insertLast(L, P);

    // INSERT FIRST: MEMASUKKAN D000 DI AWAL
    P = alokasi(K0);
    insertFirst(L, P);

    // INSERT AFTER: MEMASUKKAN D003 SETELAH D001
    ElmList* Prec = findElm(L, "D001");

    P = alokasi(K3);
    insertAfter(L, Prec, P);

    // INSERT BEFORE: MEMASUKKAN D004 SEBELUM D002
```

```
    ElmList* Succ = findElm(L, "D002");

    P = alokasi(K4);
    insertBefore(L, Succ, P);

    // MENAMPILKAN LIST SETELAH INSERT
    cout << "=== DATA SETELAH INSERT ===" << endl;
    printInfo(L);

    // FIND ELEMENT: MENCARI D001
    P = findElm(L, "D001");

    cout << "=== HASIL PENCARIAN ===" << endl;

    if (P != nullptr) {
        cout << "Nopol ditemukan: " << P->info.nopol << endl;
        cout << "Warna: " << P->info.warna << endl;
        cout << "Tahun: " << P->info.thnBuat << endl;
    } else {
        cout << "Kendaraan tidak ditemukan." << endl;
    }

    // DELETE AFTER: MENGHAPUS D003 SETELAH D001
    Prec = findElm(L, "D001");

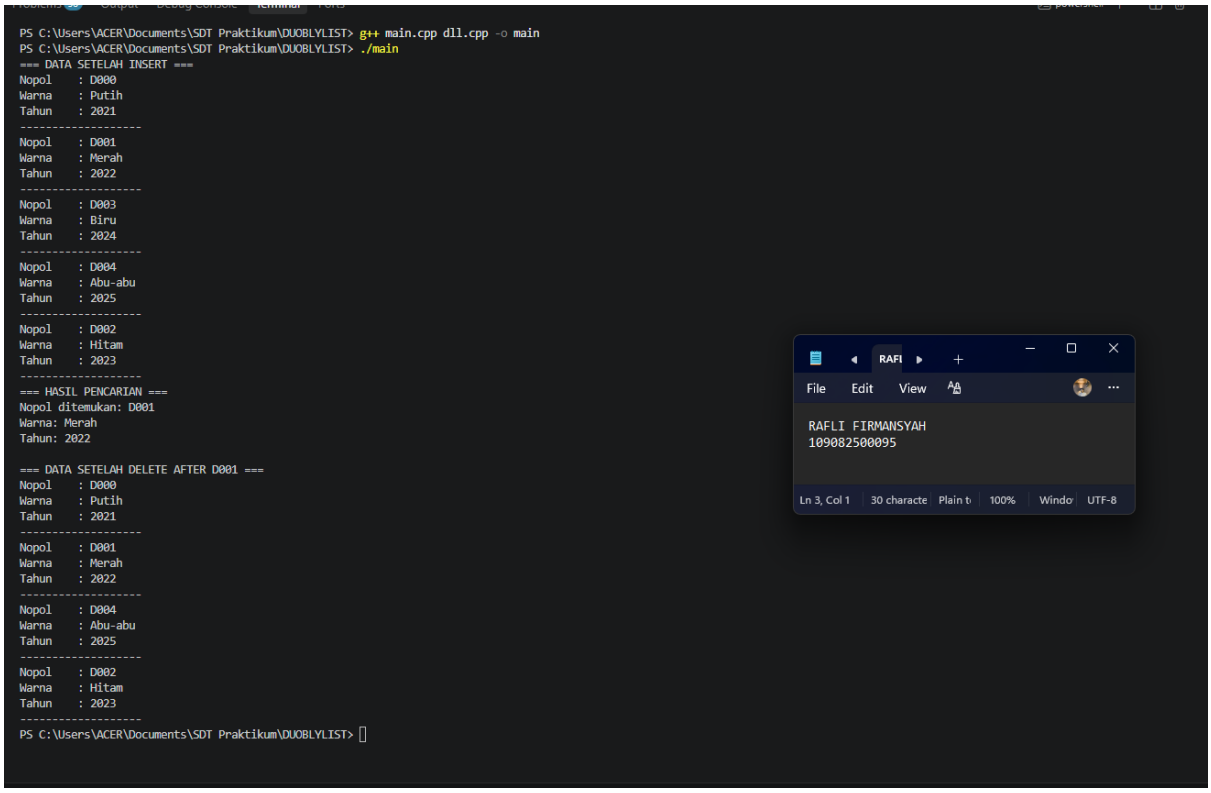
    deleteAfter(L, Prec, P);
    dealokasi(P);

    // MENAMPILKAN LIST SETELAH DELETE
    cout << endl;
    cout << "=== DATA SETELAH DELETE AFTER D001 ===" << endl;
    printInfo(L);

    // MEMBERSIHKAN SISA NODE
    while (L.first != nullptr) {
        deleteFirst(L, P);
        dealokasi(P);
    }

    return 0;
}
```

Screenshoot hasil program



Deskripsi hasil program

Program Doubly Linked List berhasil dijalankan dengan menampilkan data kendaraan yang terdiri dari nomor polisi, warna, dan tahun pembuatan. Program juga berhasil melakukan pencarian kendaraan berdasarkan nomor polisi D001 dan menghapus data kendaraan setelah D001. Hasilnya, data D001 berhasil ditemukan dan data kendaraan setelahnya berhasil dihapus dari linked list.

LATIHAN SOAL MODUL

Link Repo Github Modul X : https://github.com/raflyfirmansyah849-max/Sturktur-Data/tree/main/modul%206

4. **Soal 1**

Source Code : Doublylist.h

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>
using namespace std;

#define Nil NULL

typedef struct {
    string nopol;
    string warna;
    int thnBuat;
} infotype;

typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address First;
    address Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);

#endif
```

Source Code : Doublylist.cpp

```
#include "Doublylist.h"

void CreateList(List &L) {
    L.First = Nil;
    L.Last = Nil;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = Nil;
    P->prev = Nil;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = Nil;
}

void insertLast(List &L, address P) {
    if (L.First == Nil) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
        L.Last->next = P;
        L.Last = P;
    }
}

void printInfo(List L) {
    address P = L.Last;

    while (P != Nil) {
        cout << "no polisi  : " << P->info.nopol << endl;
        cout << "warna      : " << P->info.warna << endl;
        cout << "tahun      : " << P->info.thnBuat << endl;

        P = P->prev;
    }
}
```

Source Code : main.cpp

```
#include "Doublylist.h"

int main() {
    List L;
    infotype X;
    address P, Q;

    CreateList(L);

    for (int i = 1; i <= 4; i++) {

        cout << "masukkan nomor polisi: ";
        cin >> X.nopol;

        cout << "masukkan warna kendaraan: ";
        cin >> X.warna;

        cout << "masukkan tahun kendaraan: ";
        cin >> X.thnBuat;

        bool ditemukan = false;
        Q = L.First;

        while (Q != Nil) {
            if (Q->info.nopol == X.nopol) {
                ditemukan = true;
                break;
            }
            Q = Q->next;
        }

        if (ditemukan) {
            cout << "nomor polisi sudah terdaftar" << endl;
        } else {
            P = alokasi(X);
            insertLast(L, P);
        }

        cout << endl;
    }

    cout << "DATA LIST 1" << endl << endl;

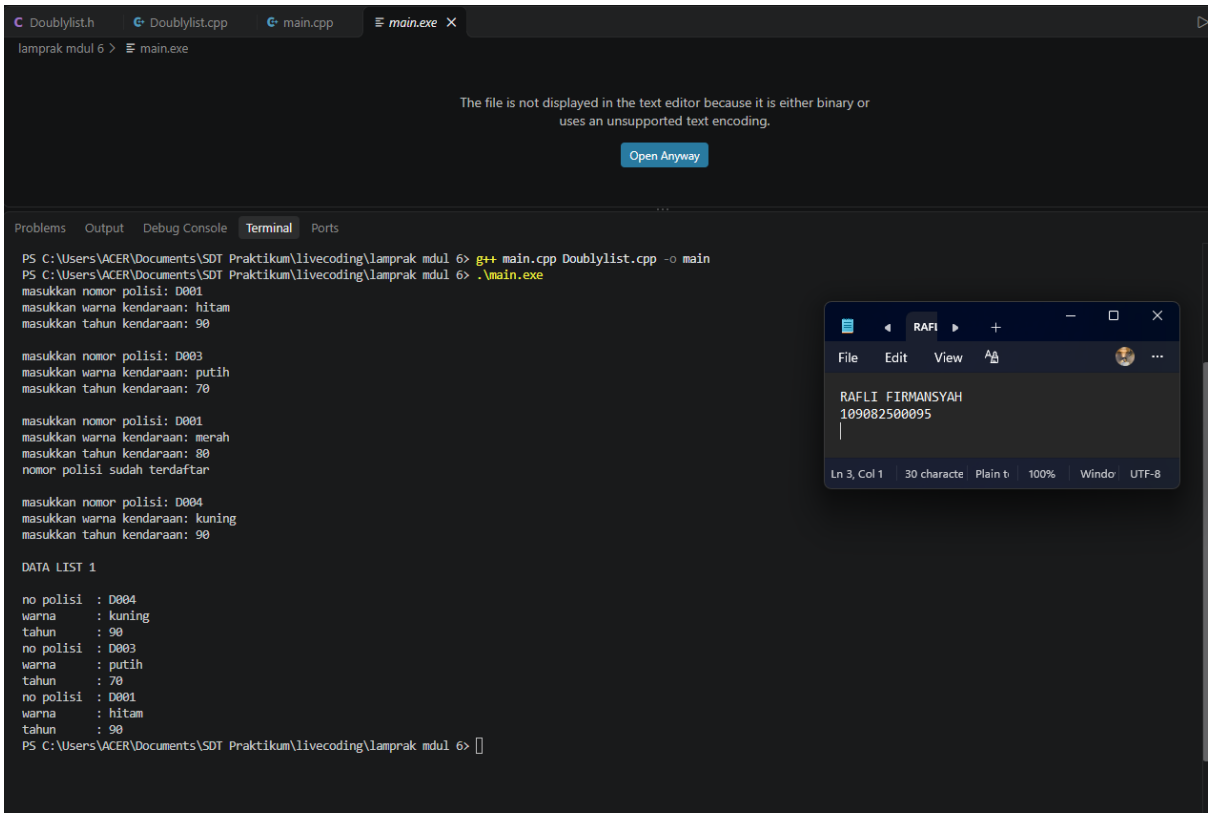
    printInfo(L);

    return 0;
}
```

Deskripsi program

Program ini menggunakan struktur data Doubly Linked List untuk menyimpan data kendaraan yang terdiri dari nomor polisi, warna, dan tahun pembuatan. Setiap elemen memiliki pointer next dan prev yang menghubungkan elemen sebelumnya dan sesudahnya. Proses penambahan data dilakukan menggunakan fungsi insertLast, sedangkan nomor polisi diperiksa agar tidak terjadi duplikasi. Data kendaraan kemudian ditampilkan dari elemen terakhir menuju elemen pertama menggunakan pointer prev.

Screenshoot Hasil Program



Deskripsi Hasil Program

Program berhasil dijalankan dengan menggunakan struktur data Doubly Linked List untuk menyimpan data kendaraan. Pada proses input, program menerima empat data kendaraan dengan nomor polisi D001, D003, D001, dan D004. Ketika nomor polisi D001 dimasukkan kembali, program mendeteksi adanya duplikasi sehingga data tersebut tidak disimpan. Setelah seluruh data dimasukkan, program menampilkan tiga data kendaraan yang berhasil tersimpan secara terbalik, yaitu dimulai dari D004, D003, hingga D001. Hal ini menunjukkan bahwa fungsi insertLast dan printInfo berjalan dengan baik.

5. Soal 2

Tambah kode di bawah pada file Doublylist.h di bagian bawah deklarasi fungsi lainnya, sebelum #endif

```
address findElm(List L, infotype x);
```

Tambah kode di bawah pada file Doublylist.cpp di bagian paling bawah file Doublylist.cpp

```
address findElm(List L, infotype x) {
    address P = L.First;

    while (P != Nil) {
        if (P->info.nopol == x.nopol) {
            return P;
        }

        P = P->next;
    }

    return Nil;
      }
```

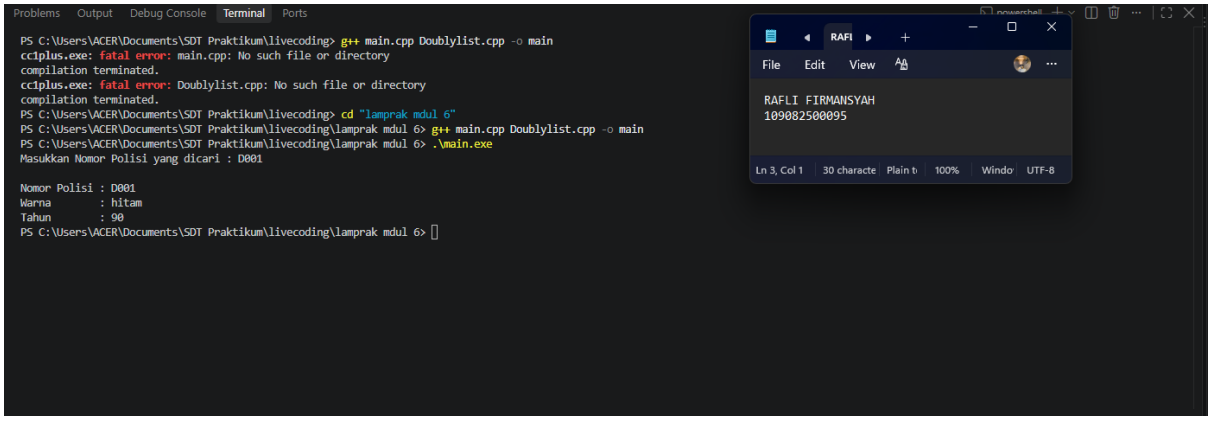
Tambah kode di bawah pada file main.cpp. yaitu ditambahkan proses pencarian setelah perintah printInfo(L); dan sebelum return 0;.

```
cout << "\nMasukkan Nomor Polisi yang dicari : ";
    cin >> X.nopol;

    Q = findElm(L, X);

    if (Q != Nil) {
        cout << "\nNomor Polisi : " << Q->info.nopol << endl;
        cout << "Warna        : " << Q->info.warna << endl;
        cout << "Tahun        : " << Q->info.thnBuat << endl;
    } else {
        cout << "Data kendaraan tidak ditemukan" << endl;
    }
```

Screenshoot hasil program



Deskripsi hasil program

Program berhasil melakukan pencarian data kendaraan menggunakan fungsi findElm() pada struktur Doubly Linked List berdasarkan nomor polisi. Ketika pengguna memasukkan nomor polisi D001, program berhasil menemukan data kendaraan dan menampilkan nomor polisi D001, warna hitam, serta tahun pembuatan 90. Hal ini menunjukkan bahwa fungsi pencarian berhasil berjalan dengan baik dan dapat menemukan elemen yang sesuai di dalam list.

6. **Soal 3**

Tambah kode di bawah pada file Doublylist.h di bawah deklarasi findElm()

```
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);
```

Tambah kode di bawah pada file Doublylist.cpp di bagian paling bawah.

```
void deleteFirst(List &L, address &P) {
    P = L.First;

    if (L.First == L.Last) {
        L.First = Nil;
        L.Last = Nil;
    } else {
        L.First = P->next;
        L.First->prev = Nil;
    }

    dealokasi(P);
}

void deleteLast(List &L, address &P) {
    P = L.Last;

    if (L.First == L.Last) {
```

```
        L.First = Nil;
        L.Last = Nil;
    } else {
        L.Last = P->prev;
        L.Last->next = Nil;
    }

    dealokasi(P);
}
void deleteAfter(address Prec, address &P) {
    P = Prec->next;

    Prec->next = P->next;

    if (P->next != Nil) {
        P->next->prev = Prec;
    }

    dealokasi(P);
}
```

Ganti seluruh kode pada file main.cpp seperti kode ini.

```
#include "Doublylist.h"

int main() {
    List L;
    infotype X;
    address P, Q;

    CreateList(L);
    X.nopol = "D001";
    X.warna = "hitam";
    X.thnBuat = 90;
    P = alokasi(X);
    insertLast(L, P);

    X.nopol = "D003";
    X.warna = "putih";
    X.thnBuat = 70;
    P = alokasi(X);
    insertLast(L, P);

    X.nopol = "D004";
    X.warna = "kuning";
    X.thnBuat = 90;
    P = alokasi(X);
    insertLast(L, P);

    X.nopol = "D003";
```

```
    Q = findElm(L, X);

    cout << "Masukkan Nomor Polisi yang akan dihapus : "
         << X.nopol << endl;

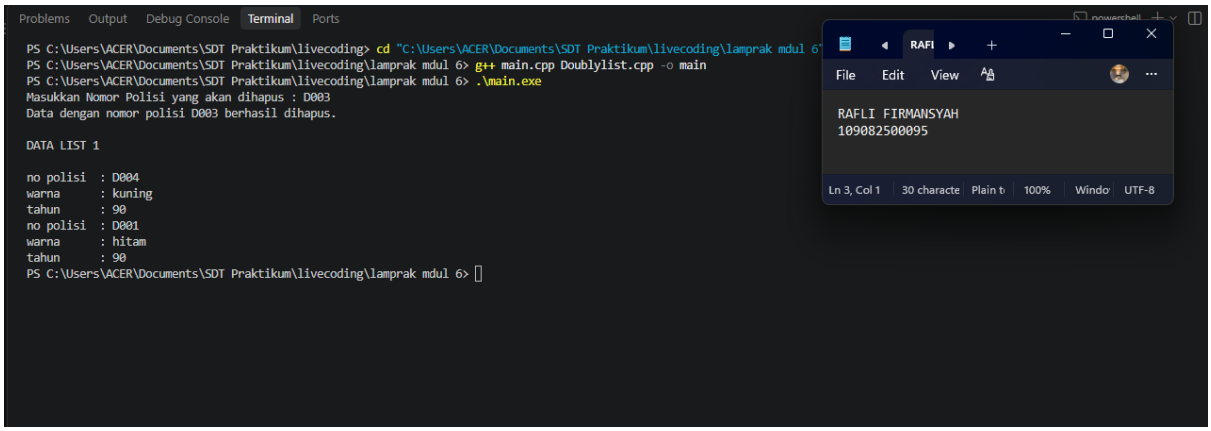
    if (Q != Nil) {
        if (Q == L.First) {
            deleteFirst(L, Q);
        } else if (Q == L.Last) {
            deleteLast(L, Q);
        } else {
            deleteAfter(Q->prev, Q);
        }

        cout << "Data dengan nomor polisi D003 berhasil dihapus."
             << endl;
    } else {
        cout << "Data kendaraan tidak ditemukan." << endl;
    }
    cout << "\nDATA LIST 1" << endl << endl;

    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi hasil program

Program berhasil menghapus data kendaraan dengan nomor polisi D003 menggunakan prosedur deleteAfter() pada struktur Doubly Linked List. Setelah data D003 ditemukan dan dihapus, program memperbarui hubungan pointer agar elemen lainnya tetap terhubung. Hasilnya, data yang tersisa adalah kendaraan dengan nomor polisi D004 dan D001. Hal ini menunjukkan bahwa prosedur penghapusan elemen berhasil berjalan dengan baik.